

把一条重试策略写清楚

从一次请求失败，到可以解释的工程决策

技术写作示例

2026 年 9 月

好的技术文章，需要让读者同时看清问题、代码和取舍。本文用一个简化的请求重试场景，展示中文正文、代码、表格、引用和插图如何放在同一篇文档里。

先定义问题

客户端偶尔收到超时错误，直接向用户报错会中断流程。但如果所有失败都立即重试，短暂故障也可能被放大。因此，我们希望重试有**明确的边界**：只处理暂时性错误，限制尝试次数，并在每次失败后等待一小段时间。

这里的 `max_attempts` 表示总尝试次数，包含第一次请求。比如设为 3，最多产生三次请求，而不是四次。

重试是一种恢复手段。只有在重复执行不会产生额外副作用时，才适合自动重试；写入类接口通常还需要幂等设计。

写下约束

- 每次调用都要有自己的超时上限。
- 只重试已约定的暂时性错误。
- 总尝试次数和累计等待时间都有界。
- 日志能区分首次调用与后续尝试。

配置项	示例值	设计意图
<code>max_attempts</code>	3	包含首次调用，避免计数歧义
<code>base_delay</code>	0.2 s	第一次失败后的基础等待
<code>max_delay</code>	2.0 s	限制单次等待的最长时间
<code>timeout</code>	1.0 s	由底层请求客户端执行

用代码表达边界

下面是一个仅依赖 Python 标准库的教学实现。调用方负责为 operation 设置请求超时，并把可重试错误映射为 TimeoutError。这一约定让重试流程保持清楚。

```
from collections.abc import Callable
from time import sleep
from typing import TypeVar

T = TypeVar("T")

def retry(operation: Callable[[], T], max_attempts: int = 3) -> T:
    if max_attempts < 1:
        raise ValueError("max_attempts 必须大于 0")

    for attempt in range(max_attempts):
        try:
            return operation()
        except TimeoutError:
            # 最后一次失败时保留原始异常，方便追踪。
            if attempt == max_attempts - 1:
                raise
            delay = min(0.2 * (2 ** attempt), 2.0)
            sleep(delay)

    raise AssertionError("unreachable")
```

把“尝试”和“等待”分开

等待只发生在相邻两次尝试之间。三次尝试最多对应两次等待，成功后立即返回。边界条件也值得单独阅读：max_attempts=1 时，只调用一次，不等待。



请求失败后，先判断是否允许重试，再决定是否等待。

指数退避的直觉

设基础等待为 b ，等待上限为 c ，第 k 次重试前的等待时间为：

$$d_k = \min(b \cdot 2^{k-1}, c)$$

这个示例没有加入随机抖动。实际系统中，多台客户端同时失败时，抖动可以让后续请求分散到不同时间点。是否需要它，应结合调用规模与服务约束判断。¹

让结果可以解释

代码之外，配置、日志和文字说明同样影响可维护性。建议把配置集中到一个明确的位置，并在日志里记录已经执行的次数。

```
retry:
  max_attempts: 3
  base_delay_seconds: 0.2
  max_delay_seconds: 2.0
  retryable_errors:
    - timeout
```

未标注语言的终端输出也应该清晰可读：

```
attempt=1 status=timeout next_delay=0.2s
attempt=2 status=ok elapsed=0.08s
result: 请求成功, 执行 2 次, 共等待 0.2 秒
```

用三个问题完成评审

1. **重复执行是否安全？** 明确请求的副作用与幂等约束。
2. **最坏情况需要多久？** 把每次请求耗时与中间等待一起计算。
3. **失败后如何定位？** 保留原始异常，并记录必要的上下文。

一篇完整的技术文章应当留下可以执行的判断依据：有哪些假设，代码做了什么，遇到边界时会怎样。清楚的层次和克制的样式，让这些信息更容易被找到。

¹ 本文代码用于演示排版与基本控制流程；未覆盖异步取消、错误分类、服务端限流等完整客户端行为。